

Planification et choix de conception

Projet INFO-F307 – Bugémon

Équipe 03

Mai 2026

Table des matières

1	Introduction	3
2	Suivi de planification	3
2.1	Organisation générale	3
2.2	Bilan des points estimés et réels	3
2.3	Évolution du périmètre	4
3	Fonctionnalités implémentées	4
3.1	H1 – Constitution d’une équipe	4
3.2	H4 – Combat automatique	4
3.3	H2 – Sauvegarde et chargement d’équipe	5
3.4	H9 – Structure de la Tour NO	5
3.5	H5 – Contrôle des actions en combat	6
3.6	H10 – Utilisation d’objets	6
3.7	H6 – Calcul de dégâts avec statistiques	6
3.8	H8 – Effet d’attaques en combat	7
3.9	H7 – Expérience et montée de niveau	7
4	Architecture générale	7
4.1	Architecture MVC-SR	7
4.2	Classes abstraites pour les écrans	9
4.3	Chargement des vues	9
4.4	Navigation globale	9
4.5	Gestion des données JSON	9
5	Choix de conception principaux	10
5.1	Modélisation des Bugémons	10
5.2	Modélisation des statistiques	10
5.3	Modélisation du combat	10
5.4	Pipeline de résolution d’un tour	11

5.5	Gestion de la Tour NO	11
5.6	Gestion de l'inventaire	11
5.7	Gestion de l'expérience	11
6	Design patterns et principes appliqués	12
6.1	Vue d'ensemble	12
6.2	Singleton	12
6.3	Factory	13
6.4	Strategy	13
6.5	Command	13
6.6	Facade	13
6.7	DTO	13
6.8	Builder	14
6.9	Principes GRASP	14
7	Prise en compte des remarques de code review	14
7.1	Séparation des couches	15
7.2	Utilisation prudente des singletons	15
7.3	Polymorphisme et réduction des conditions	15
7.4	Encapsulation et organisation du code	15
7.5	Cohérence des constantes et des exceptions	16
8	Tests et validation	16
8.1	Organisation des tests	16
8.2	Choix de testabilité	16
8.3	Limites des tests	16
9	Difficultés rencontrées et refactorisations	17
9.1	Complexité croissante du combat	17
9.2	Persistance des équipes	17
9.3	Stabilisation des modèles	17
9.4	Refactorisation de la détection de boss et de la progression	17
10	Limites connues	18
10.1	Tests graphiques	18
10.2	Évolutivité possible	18
11	Utilisation de l'IA	18
12	Conclusion	18

1 Introduction

Ce rapport présente le suivi de planification, les fonctionnalités réalisées, les choix de conception et les principales décisions architecturales prises dans le cadre du projet *Bugémon*. Le projet consistait à développer en Java une application de jeu autour d'un système de combat au tour par tour, avec constitution d'équipe, combat, progression dans la Tour NO, utilisation d'objets, calcul de dégâts avec statistiques et système d'expérience.

Le développement s'est déroulé selon une approche itérative inspirée d'Extreme Programming. À chaque itération, un certain nombre d'histoires devaient être implémenté, divisé en tâches, puis adapté l'architecture au fur et à mesure de l'augmentation de la complexité du projet.

Au début du projet, l'objectif principal était de fournir rapidement une version jouable avec une sélection d'équipe et un combat automatique. Progressivement, le projet a évolué vers une application plus complète, intégrant des actions contrôlées par le joueur, une tour de combat, un inventaire, des objets, des statistiques, des types, de l'expérience et des montées de niveau.

2 Suivi de planification

2.1 Organisation générale

Notre organisation suit l'esprit de la méthodologie XP vue au cours : le projet a été découpé en itérations courtes, chacune donnant lieu à une version fonctionnelle du logiciel. Cette approche nous a permis d'obtenir du feedback régulier, de réévaluer la complexité des histoires et d'adapter les choix de conception au fur et à mesure.

Les quatre itérations peuvent être résumées comme suit :

- **Itération 1** : constitution d'équipe et combat automatique.
- **Itération 2** : sauvegarde/chargement d'équipe et structure de la Tour NO.
- **Itération 3** : contrôle des actions en combat et utilisation d'objets.
- **Itération 4** : calcul de dégâts avec statistiques, expérience, montée de niveau et refactorisation finale.

2.2 Bilan des points estimés et réels

Le tableau suivant reprend les histoires principales de chaque itération, ainsi que les points estimés et les points réellement constatés.

Itération	Histoires principales	Points estimés	Points réels
1	H1 Constitution d'équipe, H4 Combat automatique	41	42,5
2	H2 Sauvegarder/charger une équipe, H9 Structure de la Tour NO	29	35
3	H5 Contrôle des actions en combat, H10 Utilisation d'objets	32,5	40,5
4	H6 Calcul de dégâts avec statistiques, H7 Expérience et montée de niveau, refactorisation finale	27,5	23

On observe que certaines histoires ont demandé plus de temps que prévu, notamment H5, H9 et H10. Cela s'explique par la complexité croissante de l'interaction entre le combat, la tour, les popups JavaFX, l'inventaire et la navigation entre les écrans.

L'itération 4 a également été particulière, car elle ne consistait pas seulement à ajouter de nouvelles fonctionnalités. Elle a aussi servi à stabiliser l'architecture et à améliorer certaines parties du code avant la version finale.

2.3 Évolution du périmètre

Au début du projet, les fonctionnalités étaient relativement indépendantes. La sélection d'équipe, le chargement des Bugémons depuis les fichiers JSON et le combat automatique pouvaient être développés séparément.

À partir de la troisième itération, les histoires sont devenues plus interdépendantes. Le contrôle des actions en combat dépendait du combat existant. L'utilisation d'objets dépendait à la fois du combat, de l'inventaire et de l'interface graphique. Le calcul de dégâts avec statistiques dépendait des Bugémons, des attaques, des types, des effets et du service de combat. Enfin, l'expérience dépendait de la fin de combat, de la participation des Bugémons et des statistiques modifiées lors des montées de niveau.

Cette évolution a rendu nécessaire plusieurs refactorisations. L'objectif n'était plus seulement d'ajouter des fonctionnalités, mais aussi de maintenir une architecture compréhensible, modifiable et testable.

3 Fonctionnalités implémentées

3.1 H1 – Constitution d'une équipe

La première histoire permet au joueur de constituer une équipe de Bugémons. L'écran de sélection affiche les Bugémons disponibles avec leurs noms et leurs sprites. Le joueur peut ajouter ou retirer des Bugémons de son équipe via l'interface.

Les principales règles implémentées sont :

- affichage des Bugémons disponibles ;
- ajout d'un Bugémon à l'équipe ;
- retrait d'un Bugémon sélectionné ;
- limite maximale de six Bugémons ;
- interdiction de sélectionner plusieurs fois le même Bugémon ;
- retour visuel lorsqu'un Bugémon est sélectionné.

Cette histoire a permis de mettre en place les premiers modèles du projet, notamment **Team**, **BugemonSpecie** et **TrainerBugemon**. Elle a aussi introduit les premiers éléments de l'architecture MVC utilisée dans la suite du projet.

3.2 H4 – Combat automatique

La deuxième fonctionnalité importante de la première itération est le combat automatique. Après la constitution de l'équipe du joueur, une équipe ennemie est générée automatiquement et un combat est lancé.

Les éléments principaux implémentés sont :

- génération d'une équipe ennemie via **RandomTeamFactory** ;
- affrontement entre les Bugémons actifs des deux équipes ;
- choix automatique des attaques ;

- diminution des points de vie ;
- détection des K.O. ;
- remplacement automatique d'un Bugémon K.O. ;
- affichage des sprites, noms, points de vie et messages de combat.

Cette histoire a fourni la première version du moteur de combat. Elle était volontairement simple afin de livrer rapidement un combat jouable. Les itérations suivantes ont ensuite enrichi ce moteur.

3.3 H2 – Sauvegarde et chargement d'équipe

La deuxième itération a introduit la sauvegarde et le chargement des équipes. Le joueur peut sauvegarder une équipe déjà constituée en lui donnant un nom, puis la recharger plus tard.

Les équipes sont stockées dans un fichier JSON via **TeamRepository**. Le choix de sauvegarder uniquement les identifiants des Bugémons permet d'éviter la duplication des données statiques déjà présentes dans les fichiers de base.

Les principales fonctionnalités sont :

- sauvegarde d'une équipe sous un nom choisi ;
- chargement d'une équipe sauvegardée ;
- récupération de la liste des équipes existantes ;
- vérification de l'unicité des noms d'équipes ;
- séparation entre logique métier et accès au fichier JSON.

La logique métier est gérée par **TeamService**, tandis que **TeamRepository** s'occupe uniquement de la persistance. Cette séparation respecte le principe de responsabilité unique.

3.4 H9 – Structure de la Tour NO

La structure de la Tour NO a été introduite afin de donner un objectif de progression au joueur. La tour est composée d'étages, eux-mêmes composés d'étapes de combat et de récompense.

Les classes principales sont :

- **TowerState**, qui garde l'état courant de la progression ;
- **Floor**, qui représente un étage ;
- **FloorStep**, qui représente une étape abstraite ;
- **CombatStep**, qui représente une étape de combat ;
- **RewardStep**, qui représente une étape de récompense ;
- **FloorFactory**, qui génère la structure des étages.

La progression dans la tour est gérée par **TowerService** et orchestrée par **TowerOrchestrator**, qui coordonne le lancement des combats, l'avancement entre les étapes et la navigation vers les écrans appropriés. **MetaController** expose ces opérations aux autres controllers en déléguant à **TowerOrchestrator**. De manière symétrique, **CombatOrchestrator** gère le lancement des combats libres et la navigation en fin de combat.

Deux écrans de fin ont également été ajoutés pour distinguer les différents types de victoire : un écran de déblocage d'étage (**FloorUnlockView**) et un écran de victoire finale (**TowerWinView**). Ces écrans étendent **CombatEndView** et **CombatEndController** selon la même architecture que les écrans existants.

3.5 H5 – Contrôle des actions en combat

La troisième itération a transformé le combat automatique en combat contrôlé par le joueur. À chaque tour, le joueur peut choisir une action parmi plusieurs possibilités.

Les actions disponibles sont :

- attaquer avec une des attaques du Bugémon actif ;
- changer de Bugémon ;
- utiliser un objet ;
- abandonner le combat.

Cette histoire a fortement augmenté la complexité du combat. Le moteur ne pouvait plus simplement choisir des attaques aléatoires : il devait attendre une action du joueur, gérer les menus, valider les choix, résoudre l'action du joueur et l'action ennemie, puis mettre à jour l'état du combat.

Pour éviter une méthode centrale trop complexe, les actions ont été représentées par des classes dédiées : `AttackAction`, `SwitchAction`, `UseItemAction` et `ForfeitAction`. Ces classes implémentent une interface commune `Action`.

3.6 H10 – Utilisation d'objets

L'utilisation d'objets a introduit la notion d'inventaire. Le joueur peut consulter ses objets pendant un combat et utiliser un objet, ce qui consomme son tour.

Les classes concernées sont notamment :

- `Inventory`, qui représente l'inventaire ;
- `Item`, qui représente un objet utilisable ;
- `InventoryService`, qui gère les opérations liées à l'inventaire ;
- `ItemRepository`, qui charge les objets depuis les données JSON ;
- `UseItemAction`, qui représente l'action d'utiliser un objet en combat.

Cette histoire a nécessité une attention particulière à l'interface JavaFX, car l'inventaire est affiché dans une fenêtre séparée. Il fallait donc éviter que plusieurs popups se superposent ou que l'état de la vue soit mis à jour avant la fermeture correcte de la fenêtre.

3.7 H6 – Calcul de dégâts avec statistiques

La quatrième itération a remplacé le calcul simplifié des dégâts par un calcul complet prenant en compte les statistiques, les types et les coups critiques.

Les éléments implémentés sont :

- prise en compte de l'attaque de l'attaquant ;
- prise en compte de la défense du défenseur ;
- prise en compte de l'initiative pour déterminer l'ordre d'action ;
- calcul de l'efficacité des types ;
- gestion des coups critiques ;
- affichage des types des Bugémons ;
- affichage des types des attaques ;
- affichage de l'efficacité prévue d'une attaque.

Le calcul des dégâts est isolé dans `DamageCalculator`. L'efficacité des types est représentée par `TypeEffectiveness`. Ce choix permet d'éviter de disperser les formules de calcul dans le service de combat ou dans les vues.

3.8 H8 – Effet d'attaques en combat

Bien que cette histoire n'ait jamais été demandée par le client, l'application des effets d'attaques a pu être implémentée sans aucun coût en réutilisant le système d'effet créé et mis en place dans les tâches 10 et 6, et grâce à une architecture de chargement de modèle bien prévue. En effet, lors de la désérialisation d'une `Attack`, son attribut `Attack.effects` est automatiquement peuplé d'effets utilisant le même modèle que les effets d'item (`Effect`), et donc compatibles avec l'`EffectResolver`.

Ainsi, seules deux lignes de code ont été nécessaires pour implémenter cette histoire.

3.9 H7 – Expérience et montée de niveau

L'histoire H7 ajoute une progression individuelle aux Bugémons. Après un combat remporté, les Bugémons ayant participé peuvent gagner de l'expérience. Lorsqu'un seuil est atteint, le Bugémon monte de niveau et le joueur peut choisir un bonus.

Les principales fonctionnalités sont :

- suivi des Bugémons ayant participé au combat ;
- calcul de l'expérience gagnée ;
- distribution de l'expérience entre les participants ;
- détection des montées de niveau ;
- affichage du niveau dans les écrans concernés ;
- génération de bonus de montée de niveau ;
- application du bonus choisi par le joueur.

Les classes principales sont `XP`, `XPService`, `LevelUpEvent` et `XPGainBonusFactory`. La génération des bonus proposés au joueur est assurée par `XPGainBonusFactory`, tandis que leur application est déléguée à `XPService`. Cette fonctionnalité dépend fortement du système de statistiques, car les bonus modifient les caractéristiques qui sont ensuite utilisées dans le calcul des dégâts.

4 Architecture générale

4.1 Architecture MVC-SR

Le projet suit une architecture inspirée de MVC, enrichie par des couches de services et de repositories (c.f. figure 1). L'objectif est de séparer l'interface graphique, la coordination des actions utilisateur, la logique métier et l'accès aux données.

Les vues JavaFX sont responsables de l'affichage et de la récupération des interactions utilisateur. Les controllers reçoivent ces interactions via des interfaces de type `ViewListener`. Les services contiennent la logique applicative. Les modèles représentent les entités du domaine. Les repositories centralisent l'accès aux fichiers JSON.

Cette séparation permet de limiter le couplage entre l'interface graphique et la logique métier. Par exemple, la vue de combat n'applique pas elle-même les dégâts : elle transmet le choix du joueur au `CombatController`, qui délègue ensuite au `CombatService`.

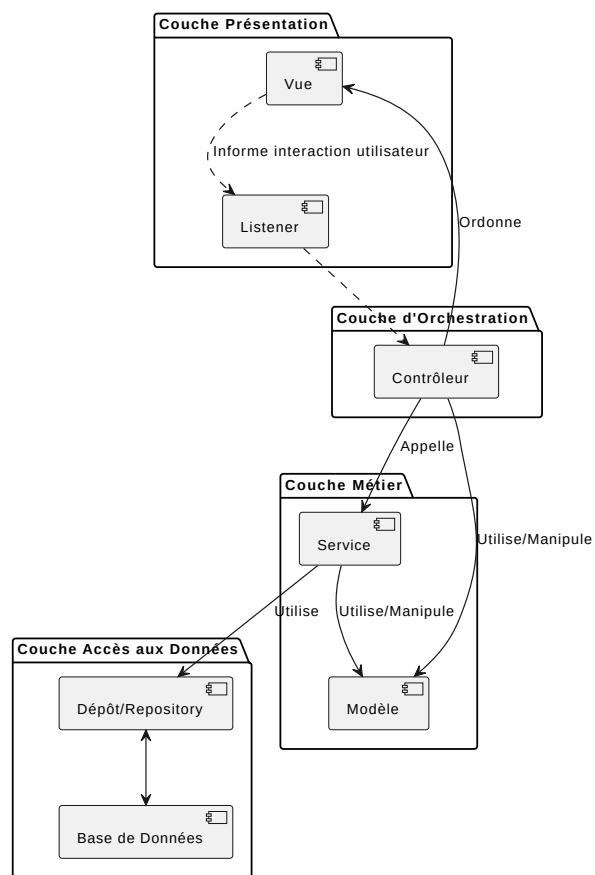


FIGURE 1 – Architecture MVC-SR utilisée pour le projet.
 Un Listener est également utilisé pour davantage de découplage entre la Vue et le Contrôleur, mais ne change en rien l'architecture elle-même.

4.2 Classes abstraites pour les écrans

Afin d'éviter de dupliquer la même structure pour chaque écran JavaFX, le projet introduit plusieurs abstractions :

- `BaseController`;
- `BaseView`;
- `BaseView.ViewListener`;
- `BaseFXMLController`.

Chaque écran possède une vue, un controller et un controller FXML. Les interactions sont transmises à travers une interface listener. Cette structure permet de garder une organisation homogène entre les écrans : sélection d'équipe, combat, inventaire, récompense, victoire intermédiaire, défaite intermédiaire et montée de niveau.

La distinction view / fxmlcontroller a deux justifications. La première est pratique, car le fxmlcontroller permet un chargement plus facile des données du fichier FXML dans la classe. Ensuite, elle permet de segmenter les affichages complexes en deux parties : la vue s'occupe de la logique **d'affichage** plus abstraite (garder trace des éléments sélectionnés, choisir de mettre à jour deux éléments en même temps sur base de la demande du contrôleur, etc.), et délègue les aspects plus concrets au fxmlcontroller, qui possède les éléments FXML appropriés (redimensionner un élément, créer un bouton d'une certaine taille, couleur, etc.).

4.3 Chargement des vues

Le chargement des vues JavaFX est centralisé dans la classe abstraite `BaseView`, notamment à travers la méthode `loadFXML`. Cette méthode charge le fichier FXML associé à une vue, récupère la racine graphique et associe le controller FXML correspondant. Ce choix évite de répéter le même code de chargement dans chaque écran. Il permet aussi de standardiser la structure des vues, en effet chaque vue JavaFX possède un fichier FXML, un controller FXML et un listener permettant de communiquer avec le controller applicatif.

4.4 Navigation globale

La navigation entre les écrans est centralisée dans `MetaController`, notamment avec la méthode `switchTo`. Cette méthode reçoit une valeur de l'énumération `Windows`, met à jour la fenêtre courante, puis exécute la commande de navigation associée. La classe `MetaController` coordonne les transitions entre les différentes fenêtres du jeu : menu principal, sélection d'équipe, combat, tour, récompense, victoire ou défaite.

Les classes `Windows`, `ScreenFactory` et `NavigationCommand` participent à cette organisation. `Windows` identifie les écrans disponibles, `ScreenFactory` centralise leur création et `NavigationCommand` permet d'associer une action de navigation à un écran.

Ce choix évite que chaque controller crée directement les autres écrans. La navigation est donc plus centralisée et plus facile à modifier.

4.5 Gestion des données JSON

Les données du jeu sont stockées dans des fichiers JSON. Les Bugémons, les attaques, les objets et les équipes sauvegardées sont chargés via des repositories.

Les principaux repositories sont :

- `BugemonRepository`;
- `AttackRepository`;
- `ItemRepository`;
- `TeamRepository`.

Ces classes isolent la logique de lecture et d'écriture des données. Les services n'ont donc pas besoin de connaître les détails du format JSON. Cela améliore la maintenabilité du code et permet de modifier le stockage sans impacter directement les controllers ou les vues.

5 Choix de conception principaux

5.1 Modélisation des Bugémons

Un choix important a été de distinguer les données statiques d'un Bugémon et les données propres à un Bugémon utilisé par un dresseur.

La classe `BugemonSpecie` représente les informations générales d'une espèce : nom, type, statistiques de base, sprite et attaques disponibles. Ces données proviennent des fichiers JSON.

La classe `TrainerBugemon` représente un Bugémon appartenant à un dresseur. Elle contient les informations qui peuvent évoluer pendant le jeu : points de vie actuels, expérience, niveau, attaques actuelles, modificateurs de statistiques et effets temporaires.

Cette séparation est importante, car plusieurs Bugémons peuvent appartenir à la même espèce tout en ayant un état différent. Par exemple, deux Bugémons de la même espèce peuvent avoir des points de vie ou un niveau différents.

5.2 Modélisation des statistiques

Les statistiques sont regroupées dans la classe `Statistics`. Elles comprennent notamment les points de vie, l'attaque, la défense et l'initiative.

Les modifications de statistiques sont représentées par des classes spécifiques comme `StatsModifieur` et `TemporaryStatsModifieur`. Cette séparation permet de distinguer les statistiques de base, les bonus permanents et les effets temporaires.

Ce choix rend le système plus extensible. Par exemple, un objet, une attaque ou une récompense de montée de niveau peuvent tous modifier des statistiques sans devoir dupliquer la logique dans plusieurs classes.

5.3 Modélisation du combat

Le combat repose sur plusieurs classes complémentaires :

- `CombatState`, qui représente l'état courant du combat ;
- `CombatService`, qui expose les opérations principales du combat ;
- `TurnEngine`, qui orchestre l'ordre des actions ;
- `AttackResolver`, qui résout les attaques ;
- `EffectResolver`, qui applique les effets ;
- `KOResolver`, qui gère les K.O. ;
- `DamageCalculator`, qui calcule les dégâts.

Cette répartition évite que **CombatService** devienne une classe trop volumineuse. Le service conserve un rôle de façade, tandis que les détails techniques du combat sont délégués à des classes spécialisées.

5.4 Pipeline de résolution d'un tour

L'une des principales évolutions architecturales concerne le pipeline de combat. Dans les premières versions, **CombatService** gérait une grande partie de la logique : choix de l'action, résolution de l'attaque, application des effets, détection des K.O. et mise à jour de l'état.

Avec l'ajout des actions contrôlées par le joueur, des objets, de l'initiative, des effets et de l'expérience, cette solution devenait trop complexe. Nous avons donc introduit un pipeline plus modulaire :

CombatController → **CombatService** → **TurnEngine** → **Action** → **AttackResolver** /
EffectResolver / **KOResolver**

CombatService reçoit l'action du joueur, prépare l'action ennemie à l'aide d'une stratégie, construit les contextes nécessaires et délègue la résolution du tour à **TurnEngine**.

TurnEngine détermine l'ordre d'action selon l'initiative des Bugémons actifs. Ensuite, chaque action s'exécute via l'interface **Action**. Une attaque est représentée par **AttackAction**, l'utilisation d'un objet par **UseItemAction**, le changement de Bugémon par **SwitchAction** et l'abandon par **ForfeitAction**.

Cette organisation évite d'avoir une grande méthode remplie de conditions dans **CombatService**. Chaque type d'action possède son propre comportement.

5.5 Gestion de la Tour NO

La Tour NO est modélisée séparément du combat. Un étage est représenté par **Floor**, et chaque étage contient une liste de **FloorStep**. Les étapes concrètes sont **CombatStep** et **RewardStep**. Le type d'une étape est représenté par l'énumération **StepType**.

La classe **FloorFactory** centralise la création des étages. Ce choix évite de répéter la structure de la tour dans plusieurs parties du code. Si la structure d'un étage devait changer, la modification serait localisée dans cette factory.

TowerState garde l'état de progression du joueur dans la tour : étage courant, étape courante et informations liées à la victoire ou à la défaite. **TowerService** gère l'avancement dans la tour.

5.6 Gestion de l'inventaire

L'inventaire est représenté par la classe **Inventory**. Les objets sont représentés par **Item** et chargés via **ItemRepository**. La logique d'utilisation est gérée par **InventoryService** et par l'action **UseItemAction** lorsqu'un objet est utilisé pendant un combat.

Ce choix permet de garder l'inventaire indépendant de l'interface graphique. La vue affiche les objets disponibles, mais elle ne décide pas elle-même de leurs effets.

5.7 Gestion de l'expérience

L'expérience est modélisée par la classe **XP**. La distribution de l'expérience après un combat est gérée par **XPService**. Lorsqu'un Bugémon monte de niveau, un **LevelUpEvent** est créé afin de transmettre l'information à l'interface.

Les bonus de niveau sont générés via `XPGainBonusFactory` et appliqués à l'aide de `XPService`. Cette séparation permet de distinguer clairement trois responsabilités :

- calculer l'expérience gagnée ;
- détecter les montées de niveau ;
- appliquer le bonus choisi.

6 Design patterns et principes appliqués

6.1 Vue d'ensemble

Nous avons utilisé plusieurs design patterns et principes de conception afin de garder un code plus modulaire et plus facile à maintenir.

Pattern	Classes concernées	Justification
Singleton	<code>BugemonRepository</code> , <code>AttackRepository</code> , <code>ItemRepository</code> , <code>TeamRepository</code>	Garantir un accès centralisé aux données JSON et éviter de multiplier les lectures ou les états incohérents.
Factory	<code>RandomTeamFactory</code> , <code>TeamFactory</code> , <code>FloorFactory</code> , <code>ScreenFactory</code> , <code>XPGainBonusFactory</code>	Isoler la création d'objets complexes : équipes adverses, équipes sauvegardées, étages de la tour, écrans JavaFX et bonus de montée de niveau.
Strategy	<code>AttackChoiceStrategy</code> , <code>RandomOffenseStrat</code> , <code>ReplacementStrategy</code> , <code>FirstAliveStrategy</code>	Séparer la logique de décision du moteur de combat. Cela permettrait d'ajouter d'autres comportements ennemis sans modifier le moteur.
Command	<code>Action</code> , <code>AttackAction</code> , <code>UseItemAction</code> , <code>SwitchAction</code> , <code>ForfeitAction</code>	Encapsuler chaque action de combat dans un objet exécutable, au lieu de multiplier les conditions dans <code>CombatService</code> .
Facade	<code>CombatService</code> , <code>TeamService</code> , <code>TowerService</code> , <code>InventoryService</code>	Fournir aux controllers une interface simple vers une logique métier plus complexe.
DTO	<code>CombatStateDTO</code> , <code>AttackWithEffectivenessDTO</code>	Transmettre à la vue uniquement les données nécessaires à l'affichage, sans exposer tout le modèle métier.
Builder	<code>TrainerBugemon.Builder</code>	Faciliter la création de Bugémons possédant plusieurs attributs et éviter les constructeurs trop longs.

6.2 Singleton

Les repositories sont implémentés sous forme de singletons. Ce choix est adapté car les données chargées depuis les fichiers JSON sont communes à toute l'application. Il n'est pas nécessaire de créer plusieurs instances de `BugemonRepository`, `AttackRepository` ou `ItemRepository`.

Le Singleton permet donc :

- un point d'accès global aux données ;

- une lecture centralisée des fichiers ;
- une réduction du risque d'incohérence entre plusieurs instances.

6.3 Factory

Plusieurs factories sont utilisées dans le projet. **RandomTeamFactory** génère les équipes adverses. **TeamFactory** participe à la création d'équipes à partir de données sauvegardées. **FloorFactory** crée la structure des étages. **ScreenFactory** centralise la création des écrans JavaFX. **XPGainBonusFactory** génère les bonus proposés lors des montées de niveau.

Ce pattern est utile lorsque la création d'un objet est suffisamment complexe pour ne pas être dispersée dans le reste du code. Il permet aussi de respecter une meilleure cohésion : la logique de création est regroupée dans une classe dédiée.

6.4 Strategy

Le pattern Strategy est utilisé pour les comportements qui peuvent varier. Par exemple, **AttackChoiceStrategy** définit la manière dont un adversaire choisit son attaque. L'implémentation actuelle principale est **RandomOffenseStrat**, qui choisit une attaque de manière aléatoire.

Le même principe est utilisé pour le remplacement d'un Bugémon K.O. avec **ReplacementStrategy** et **FirstAliveStrategy**. Cette organisation permettrait d'ajouter plus tard une intelligence artificielle plus avancée sans modifier directement le moteur de combat.

6.5 Command

Les actions de combat sont représentées par des objets qui implémentent l'interface **Action**. Chaque action possède une méthode d'exécution et reçoit un **ActionContext**.

Les classes concernées sont :

- **AttackAction**;
- **UseItemAction**;
- **SwitchAction**;
- **ForfeitAction**.

Ce choix évite d'avoir un grand bloc conditionnel dans **CombatService**. Chaque action porte sa propre logique. Le moteur de tour n'a donc pas besoin de connaître les détails de chaque action : il appelle simplement **execute**.

6.6 Facade

Les services jouent un rôle de façade. Par exemple, **CombatController** interagit principalement avec **CombatService**. Il n'a pas besoin de manipuler directement **AttackResolver**, **EffectResolver**, **KOResolver** ou **TurnEngine**.

De même, **TeamSelectionController** passe par **TeamService** pour ajouter, retirer, sauvegarder ou charger une équipe. Cette organisation simplifie les controllers et réduit le couplage entre l'interface et le modèle.

6.7 DTO

Les DTO permettent de transmettre à la vue uniquement les données nécessaires à l'affichage. Par exemple, **CombatStateDTO** contient les informations utiles pour afficher le combat : noms des

Bugémons, points de vie, niveaux, sprites, types et messages.

`AttackWithEffectivenessDTO` permet d'associer une attaque à son efficacité prévue. Cela évite que la vue doive connaître les détails du calcul d'efficacité des types.

Ce choix limite le couplage entre la vue et le modèle métier. La vue n'a pas besoin de manipuler directement les objets internes du combat.

Cependant, tous les modèles ne sont pas passés à la vue en DTO. Les modèles immutables, comme les records ou enums, et une classe possédant uniquement des getters (`BugemonSpecie`, qui n'est pas un record à cause du design pattern builder) ne possèdent pas de DTO. Créer un `BugemonSpecieDTO` à l'avenir pourrait être pertinent pour empêcher explicitement l'accès à des méthodes non-getter, mais cela n'a pas été une priorité pour l'équipe car il n'est pas attendu que le modèle devienne mutable à l'avenir (application du principe YAGNI).

6.8 Builder

La classe `TrainerBugemon` utilise un Builder. Ce choix est justifié par le nombre d'informations nécessaires à la création d'un Bugémon de dresseur : espèce, surnom, points de vie, attaques, expérience, statistiques et effets.

Un constructeur classique avec beaucoup de paramètres serait difficile à lire et plus propice aux erreurs. Le Builder permet une création plus claire et plus flexible.

6.9 Principes GRASP

Plusieurs principes GRASP ont guidé l'architecture.

Haute cohésion Les responsabilités sont réparties entre plusieurs classes spécialisées. Par exemple, le calcul des dégâts est dans `DamageCalculator`, l'application des effets dans `EffectResolver`, la résolution des attaques dans `AttackResolver` et la progression de la tour dans `TowerService`.

Couplage faible Les vues ne manipulent pas directement les repositories ou les détails du modèle. Elles passent par les controllers et les services. Cela permet de modifier la logique métier sans devoir réécrire l'interface graphique.

De plus, certaines méthodes retournaient directement les listes internes. Ces méthodes retournent désormais une vue non modifiable via `Collections.unmodifiableList`, ce qui empêche l'appelant de modifier l'état interne sans passer par les méthodes prévues à cet effet.

Contrôleur Les controllers reçoivent les actions de l'utilisateur après l'interface graphique. Ils jouent le rôle de point d'entrée pour les opérations système, puis délèguent aux services.

Créateur Les factories sont responsables de créer certains objets complexes, comme les étages ou les équipes adverses. Cela évite de disperser la logique de construction dans plusieurs classes.

7 Prise en compte des remarques de code review

Les séances de code review ont permis d'identifier plusieurs points d'amélioration dans notre base de code. Certaines remarques ont été directement prises en compte pendant les itérations suivantes, tandis que d'autres constituent encore des pistes d'amélioration.

7.1 Séparation des couches

Une remarque importante concernait les violations possibles entre les couches de l'architecture. Dans notre projet, l'objectif est que les vues et les controllers ne manipulent pas directement les repositories. L'accès aux données doit passer par les services, afin de conserver une séparation claire entre interface graphique, logique applicative et persistance.

Cette remarque a renforcé notre choix d'utiliser des services comme `TeamService`, `CombatService`, `TowerService` ou `InventoryService`. Ces classes jouent le rôle d'intermédiaires entre les controllers et les modèles ou repositories. Il reste cependant des classes centrales, comme `MetaController`, qui connaissent beaucoup d'éléments du système. Cela constitue une piste d'amélioration.

7.2 Utilisation prudente des singletons

Nous avons utilisé le pattern Singleton pour plusieurs repositories, comme `BugemonRepository`, `AttackRepository`, `ItemRepository` et `TeamRepository`. Ce choix permet de centraliser l'accès aux fichiers JSON et d'éviter de relire plusieurs fois les mêmes données.

Cependant, les remarques de code review ont rappelé qu'un excès de singletons peut rendre le code proche d'un ensemble de variables globales. Cela augmente le couplage et rend certaines dépendances moins visibles. Dans notre projet, ce choix reste un compromis pratique, mais une amélioration possible serait d'injecter explicitement les repositories dans les services qui en ont besoin.

7.3 Polymorphisme et réduction des conditions

Les remarques de code review ont aussi insisté sur l'utilisation du polymorphisme pour éviter les grands blocs conditionnels. Cette idée est visible dans notre moteur de combat avec l'interface `Action` et ses différentes implémentations : `AttackAction`, `UseItemAction`, `SwitchAction` et `ForfeitAction`. Grâce à cette structure, le moteur de combat peut exécuter une action sans devoir connaître tous les détails de son type concret. Cela rend le code plus extensible et limite les `if` ou `switch` dans `CombatService`.

Le même principe s'applique aux événements de combat : la classe abstraite `CombatEvent` impose à chaque sous-classe, comme `AttackEvent` ou `KoEvent`, d'implémenter `toString()` avec son propre message. Le `CombatRenderer` peut ainsi afficher le journal de combat en appelant simplement `event.toString()` sur chaque événement, sans aucun `switch`.

7.4 Encapsulation et organisation du code

D'autres remarques concernaient l'encapsulation et l'organisation générale du code. Par exemple, il est préférable de mettre `final` lorsque c'est possible, de limiter les méthodes `static`, de retourner des collections non modifiables lorsque l'appelant ne doit pas les modifier, et d'éviter les packages trop volumineux.

Ces remarques ont servi de repères pour améliorer progressivement la qualité du code. Certaines sont déjà partiellement appliquées, tandis que d'autres restent des pistes d'amélioration. En particulier, certains packages pourraient encore être mieux découpés en sous-packages, et certaines listes internes pourraient être mieux protégées contre les modifications externes.

7.5 Cohérence des constantes et des exceptions

Les séances de review ont conduit à plusieurs corrections transversales. L'utilisation de `MessageConstants` et `LabelConstants` a été standardisée avec un import direct dans l'ensemble des fichiers concernés. Plusieurs chaînes hardcodées ont été extraites vers `MessageConstants` et les constantes inutilisées ont été supprimées (YAGNI).

Les usages trop génériques de `RuntimeException` et `NullPointerException` ont été remplacés respectivement par `IllegalStateException` et `IllegalArgumentException`. L'idiome `Objects.requireNonNull` a également été introduit dans `CombatService`. Enfin, les méthodes `@Deprecated` de `CombatService` et le champ `inventory` de `Team`, jamais initialisé, ont été supprimés.

8 Tests et validation

8.1 Organisation des tests

Les tests unitaires couvrent principalement les modèles, les services et les repositories. Ces éléments peuvent être testés indépendamment de JavaFX, ce qui rend les tests plus stables et plus faciles à automatiser.

Les tests portent notamment sur :

- la constitution d'équipe avec `TeamTest` et `TeamServiceTest` ;
- les modèles partagés avec `BugemonSpecieTest`, `TrainerBugemonTest`, `StatisticsTest`, `StatsModifierTest` et `TemporaryStatsModifierTest` ;
- les modèles de dresseur avec `EnemyTest`, `PlayerTest` et `CombatStateTest` ;
- la génération d'équipe avec `RandomTeamFactoryTest` et `BugemonServiceTest` ;
- le chargement des données avec `BugemonRepositoryTest`, `AttackRepositoryTest`, `ItemRepositoryTest` et `TeamRepositoryTest` ;
- le calcul des dégâts avec `DamageCalculatorTest` ;
- l'efficacité des types avec `TypeEffectivenessTest` ;
- la résolution du combat avec `CombatServiceTest`, `TurnEngineTest`, `AttackResolverTest`, `EffectResolverTest`, `KOResolverTest` et `RandomOffenseStratTest` ;
- l'inventaire avec `InventoryTest` et `InventoryServiceTest` ;
- la progression dans la tour avec `TowerStateTest`, `FloorTest`, `FloorStepTest`, `TowerServiceTest` et `FloorFactoryTest` ;
- l'expérience avec `XPServiceTest` et `XPGainBonusFactoryTest`.

8.2 Choix de testabilité

Un objectif important était de placer le maximum de logique hors des classes JavaFX. Les vues sont plus difficiles à tester automatiquement, alors que les modèles et services peuvent être testés avec JUnit.

En effet, la vue affiche les informations, mais la logique métier reste dans les services. Par exemple, l'efficacité d'une attaque n'est pas calculée dans la vue ; elle est fournie par le service sous forme de DTO.

8.3 Limites des tests

Les vues JavaFX et les popups ont été principalement validés manuellement. Cela concerne notamment les fenêtres d'inventaire, de choix de Bugémon, de récompense et de montée de

niveau.

Cette limite est partiellement compensée par la séparation entre interface et logique métier. Les comportements importants sont testés dans les services, tandis que les vues sont utilisées pour l’affichage et les interactions utilisateur.

9 Difficultés rencontrées et refactorisations

9.1 Complexité croissante du combat

La difficulté principale du projet a été l’augmentation progressive de la complexité du combat. En effet, l’ajout des actions du joueur, des objets, de l’initiative, des effets, des statistiques et de l’expérience a rendu la logique beaucoup plus complexe.

La première solution consistait à gérer une grande partie de cette logique dans `CombatService`. Cette approche est devenue difficile à maintenir. La refactorisation a consisté à extraire plusieurs responsabilités vers `TurnEngine`, `AttackResolver`, `EffectResolver`, `KOResolver` et les classes d’actions.

9.2 Persistance des équipes

La sauvegarde des équipes a demandé une réflexion sur ce qui devait être stocké. Sauvegarder tous les objets complets aurait dupliqué des informations déjà présentes dans les fichiers JSON. Nous avons donc choisi de sauvegarder principalement les identifiants nécessaires pour reconstruire une équipe.

Ce choix rend la sauvegarde plus légère et plus cohérente avec le rôle des repositories.

9.3 Stabilisation des modèles

Au début du projet, la distinction entre espèce de Bugémon, Bugémon appartenant à un dresseur et Bugémon actif en combat n’était pas immédiate. Cette difficulté a conduit à clarifier progressivement les responsabilités entre `BugemonSpecie`, `TrainerBugemon`, `Team` et `CombatState`.

Cette clarification a ensuite facilité l’ajout des statistiques, de l’expérience et des effets temporaires.

9.4 Refactorisation de la détection de boss et de la progression

La détection du type d’étape boss reposait sur deux mécanismes parallèles : un champ booléen dans `CombatStep` et la valeur `StepType.BOSS` dans l’énumération `StepType`. Ces deux sources encodaient la même information. La méthode `isBoss()` a donc été supprimée ; les appelants utilisent désormais directement `getStepType() == StepType.BOSS`. Cette simplification a aussi permis de corriger `CombatController`, qui lisait le statut boss depuis `CombatState` (donnée de présentation) plutôt que depuis `TowerState` (source métier).

Par ailleurs, la méthode `resetProgression()` a été ajoutée à `TrainerBugemon` et à `Team` pour remettre à zéro l’expérience et le niveau lors du démarrage d’une nouvelle run. `TowerOrchestrator` appelle `resetProgression()` au lancement de la tour, tandis que `resetForCombat()` est conservé pour les transitions entre combats au sein d’une même run, ces deux opérations ayant des périmètres distincts.

10 Limites connues

Certaines limites restent présentes dans la version finale du projet.

10.1 Tests graphiques

Les vues JavaFX ne sont pas couvertes par des tests automatiques complets. Les interactions graphiques ont été validées manuellement. Une amélioration possible serait d'ajouter des tests d'interface avec un outil spécialisé pour JavaFX.

10.2 Évolutivité possible

L'architecture actuelle permettrait cependant d'ajouter certaines fonctionnalités plus facilement. Par exemple, une nouvelle stratégie ennemie pourrait être ajoutée en implémentant `AttackChoiceStrategy`. De nouveaux objets pourraient être ajoutés via les données JSON et le repository d'objets. De nouvelles structures d'étage pourraient être générées en modifiant ou en étendant `FloorFactory`.

11 Utilisation de l'IA

L'intelligence artificielle a été utilisée comme outil d'aide ponctuel pendant le projet. Elle n'a pas remplacé la conception ou l'implémentation réalisées par l'équipe, mais a servi de support pour certaines tâches.

Les usages principaux ont été :

- aide à la rédaction de la JavaDoc ;
- aide à la compréhension de problèmes JavaFX ;
- aide à la formulation de certaines explications de conception ;
- aide à l'identification de pistes de refactorisation ;

12 Conclusion

Au terme des quatre itérations, le projet est passé d'une première version centrée sur la constitution d'équipe et le combat automatique à une application plus complète intégrant la sauvegarde, la Tour NO, les actions contrôlées par le joueur, l'utilisation d'objets, les statistiques, les types, l'expérience et les montées de niveau.

La principale difficulté a été l'augmentation progressive de la complexité du combat. Les premières versions fonctionnaient pour un combat automatique simple, mais l'ajout des choix du joueur, des objets, de l'initiative, des effets et de l'expérience a nécessité une architecture plus modulaire.

Les refactorisations réalisées ont permis de mieux répartir les responsabilités entre les services, les modèles, les resolvers, les factories, les repositories et les vues. Les patterns comme Factory, Strategy, Command, Facade, DTO et Singleton ont contribué à rendre le code plus maintenable et plus extensible.

Le projet illustre donc l'intérêt d'une approche itérative : livrer rapidement une version fonctionnelle, obtenir du feedback, puis améliorer progressivement le code et l'architecture en fonction des nouvelles histoires et des difficultés rencontrées.